

Porting the TL/1 compiler to the 6809 board microcontroller.

 flexonsbd.blogspot.com/2021/05/6809tl1.html

Sunday, May 9, 2021

I know it's a bit late, but I've ported the old TL/1 compiler for the 6800 to my homemade 6809SBC.

The TL/1 compiler is a compiler for the 6800 created by Hiroshi Onishi in 1980. Its distinguishing feature is that its variables are only 1 byte (8 bits!), making it a compact, one-pass compiler with a size of just over 4kB. Like the GAME language that Onishi created earlier, its source code was publicly available, so it was ported to various personal computers. I also used the TL1-FM version ported to the FM-7, but I stopped using it as much after switching to the FM77.

I've always wanted to create my own programming language someday, and I've been thinking about various ideas (though I'm not making any progress...). I thought that creating a language with 1-byte variables would be relatively easy, so I started by porting TL/1 for the 6800 to the 6809 to gain some concrete knowledge about compiler creation. However, it ended up taking me almost a month.

I'm writing down the whole story here in the hope that it might be of some help to someone.

The transplant procedure was carried out as follows:

(1) Replace the 6800 instruction codes in the source program with 6809 instruction codes.

Many instructions can be simply replaced, but it's important to note that the way the stack is used differs between the 6800 and 6809. In particular, parts of the code that use the stack for calculations will need to be rewritten. Also, in some cases, the CPX instructions (\$8C, \$9C) are used to skip the next instruction rather than for calculations.

(2) Replace the compiler-generated code from the 6800 version to the 6809 version.

While it's said that the 6800 and 6809 are not binary-level compatible, there are quite a few instructions that share the same instruction code. Even among the differences, those that are easily identifiable, like LDX (\$DE → \$9E), are fine, but there are also instructions that have similar functions but differ in some parts, which is troublesome. (See below)

This is a ported 6809 board. This is the board I introduced in my blog post on October 15, 2018, titled "6809/6802 Dual-Purpose Board Completed," and it allows you to switch between 6809 and 6802 using a switch. When switching, the memory is not cleared, so data created with one CPU can be used with the other CPU. It also has two ACIA chips, one for the console and the other for data input/output, both operating at 38400 Baud.

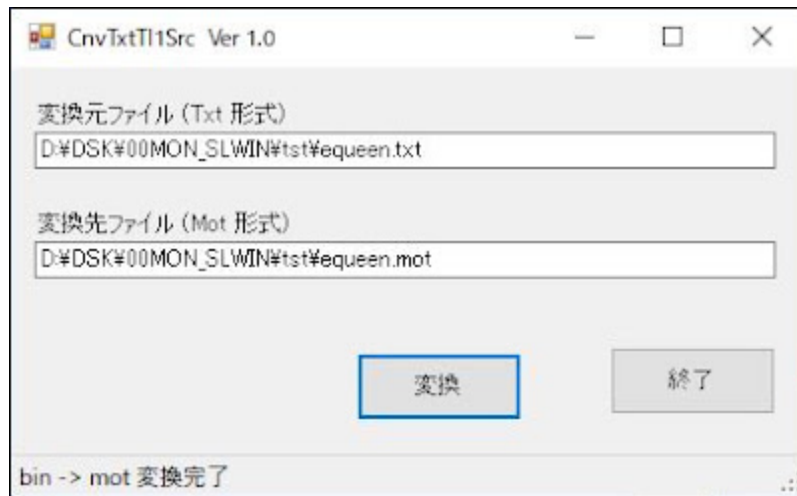


Ported 6809/6802 DualBoard

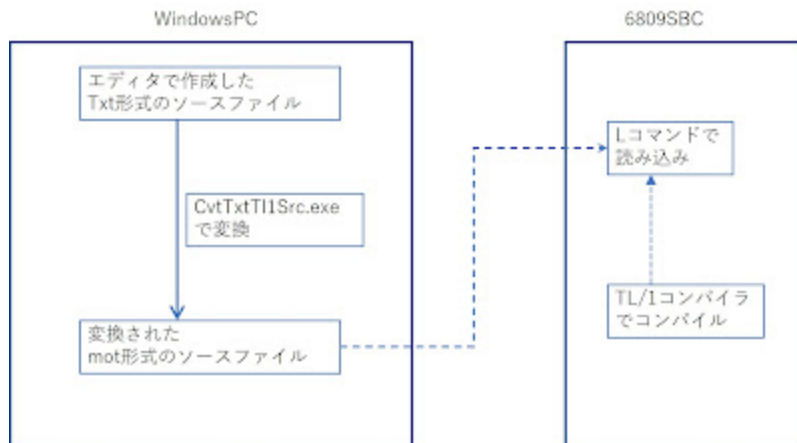
Specifically, we proceeded with the following steps:

As preparation, the original TL/1 compiles assuming that source files created with GAME are stored starting from \$3000. However, launching GAME every time is troublesome, and adding line numbers is also cumbersome, so I created a software called CvtTxtTl1Src.exe that allows you to create source code in a Windows editor, convert it to GAME format, and then convert it to mot format. Its functions include adding line numbers, converting those line numbers to binary, converting line ending codes to \$00, adding \$FF to the end of the file, and converting to mot format.

As shown in the diagram below, you can obtain the conversion result simply by dragging and dropping the file.



CvtTxtTl1Src



Work procedure

- (1) Load the source code in mot format into \$3000 using the L command on the monitor installed in the custom-built 6809SBC.
- (2) Load the prototype compiler converted to mot format into \$1000.
- (3) Use the G command on the monitor to execute from \$1000 and create an object.
- (4) If an error occurs during generation, check the compiler source and correct any relevant parts. After correcting, repeat from (2).
- (5) Once the object is successfully created, try running it from \$2000 using the G command. If you encounter errors or the results are incorrect, correct the source code and repeat from (2).

(6) The process is complete when the intended results are obtained.

Following these steps, we sequentially checked things like assigning values to variables, verifying control structures, and passing data between functions and procedures. However, the bug that caused us the most trouble until the very end was that functions and procedures would behave strangely if they had two or more local variables.

The 6809SBC I used is a dual board with both 6802 and 6809 chips. I created objects using the 6800TL/1, which I knew worked correctly, and then repeatedly checked by comparing them with those created using the 6809TL/1. For example, the 8-QUEEN object from the original TL/1 article was created identically on both the 6802 and 6809, yet it worked correctly on the 6802 but hung up on the 6809. Of course, I confirmed that the runtime routines were working correctly.

This situation continued for about one or two weeks, and I was getting quite fed up and thinking about giving up, but my SBC has a custom-made monitor, as well as MIKBUG2 on the 6802 and ASSIST09 on the 6809. So I rewrote the TL/1 input/output to match ASSIST09 and switched to ASSIST09, and by using its trace command to dump and check the work area and object area after executing each command, I was finally able to pinpoint the cause of the bug.

The bug was caused by the index instruction STA n,X. In the 6800, n is in the 8-bit range, whereas in the 6809, n in the range of 0 to 7F is a 5-bit instruction. TL/1 makes extensive use of code that modifies n in STA n,X during execution, but for example, if you increase n from 0 to 1, it will be a positive value up to 15, but after that it will become a negative value of -16.

The 6809 has an accumulator offset instruction,

Instructions in 6800 → Instructions in 6809

(5 bytes total for 2 instructions) (2 bytes per instruction)

STB xxxx (xxxx is the address of n) STA B,X

STA n,X

This allowed us to replace the 5-byte instruction with a 2-byte instruction, thus resolving the bug.

We haven't finished checking all the built-in functions and such yet, but so far we haven't found any strange behavior.

Nevertheless, while it's true that compact code can be generated by rewriting object code during execution, it's also pleasing to recall the criticism that this aspect prevented it from being ROM-based. Therefore, it's fortunate that this issue was inadvertently resolved by replacing it with 6809-specific instructions. Furthermore, if the generated code, including the runtime routines from \$2000 onwards, can be made position-independent, then running it on OS-9 might become a realistic possibility.

Through this porting process, my understanding of how to generate objects for control structures and other elements has improved.

However, the ASSIST09 trace command is a very powerful tool. I would like to implement it on my monitor if possible.

My ultimate goal is to implement OS-9 Level 1 on a 6809SBC and run languages like TL/1 on it, but I feel like I've made a small step forward.

I believe Hiroshi Onishi is a university professor specializing in the humanities, but I'm truly impressed by the originality and creativity behind GAME and TL/1.

The following two books are my regular references.

- MC6809-MC6809E Microprocessor Programming Manual (Motorola)

6809 Assembly Programming (by Asao Suenaga, Science Co., Ltd.)

In addition, this book was very helpful. The explanation of object creation methods, in particular, was very useful for understanding TL/1.

- Techniques for Creating Original Compilers for New Languages Compatible with Z80 CPUs (by Hiroyuki Onuki, MIA Corporation)